7.4.2 Ensuring disposability: Graceful shutdown

Having applications start quickly is not enough to address our scalability needs. Whenever an application instance is shut down, it must happen gracefully without clients experiencing downtime or errors. Gracefully shutting down means the application stops accepting new requests, completes all those still in progress, and closes any open resources, like database connections.

All the embedded servers available in Spring Boot support a graceful shutdown mode, but in slightly different ways. Tomcat, Jetty, and Netty stop accepting new requests entirely when the shutdown signal is received. On the other hand, Undertow keeps accepting new requests but immediately replies with an HTTP 503 response.

By default, Spring Boot stops the server immediately after receiving a termination signal (`SIGTERM`). You can switch to a graceful mode by configuring the `server.shutdown` property. You can also configure the *grace period*, which is how long the application can spend processing all the pending requests. After the grace period expires, the application is terminated even if there are still pending requests. By default, the grace period is 30 seconds. You can change it through the `spring.lifecycle.timeout-per-shutdown-phase` property.

Let’s configure graceful shutdown for Catalog Service. We could do that via environment variables or set it as a default configuration. We’ll go with the second option. Open the application.yml file located in the catalog-service/src/main/resources folder, and update the configuration as follows:

```
server:
  port: 9001
```